Typed SNApps: an illustrated walkthrough

A tour of the SRNet client · October 2026

SRNet is a small network with a simple habit: check the signature first, trust the content second. Typed SNApps are where that habit becomes something you can use every day. A message board, a chat room, a wiki, a store, a file drop, a directory of people, a directory of communities and your mail all live inside your own client, and the hosts that carry them serve only signed or sealed data. This walkthrough follows the client screen by screen, from the first page to joining rooms on other hosts, sending mail and paying for an exit, and ends with how to run a host of your own.

SRNet is an independent project. It is not affiliated with or endorsed by the Session Foundation or the Session Technology Foundation.

1. What typed SNApps are

On most networks a site sends you a page full of its own code, and your browser runs it. Over an anonymity network that is a poor bargain. A hostile page can try to fingerprint you, phone home, or quietly change what it shows.

Typed SNApps turn this around. The SRNet client carries a small set of **reviewed templates**: one for a board, one for a chat room, one for a wiki, a profile, a store, a file drop and a directory. A `.sesh` host sends only **signed records** of known types: posts, room lines, wiki revisions, listings, file entries, profiles, directory entries. The client checks every signature, keeps the records locally and draws them with a template it already trusts. Mail works the same way with one more layer: each message is signed by its sender and sealed to its recipient, so the host that carries it can read neither.

So the host never runs code on your machine, never decides what your screen looks like, and cannot quietly change a record. A mirror can serve the same records as a second copy, because each record proves itself. Plain-HTML `.sesh` sites still open, in a separate **locked-down viewer** with no scripts and no remote loads.

2. The client at a glance

One app covers all of SRNet: typed SNApps, mail, the exit marketplace and your SRNet balance.

The install steps are written for a fresh Ubuntu 24.04 desktop (amd64), the supported platform for this release. The full commands are in the **Install** section of the SRNet-Desktop README, in four steps:

1. **Session Router.** Install the Session Foundation's `session-router-client` service with the SRNet build of the router, which carries all `.sesh` traffic.
2. **`.sesh` names.** A one-time setting so Ubuntu's resolver sends `.sesh` lookups to Session Router.
3. **The SRNet client.** Clone the repository and run `./install.sh`. No root is needed. Everything goes under `~/.local/share/srnet-client`, `srnet` and `srnet-relayer` land in `~/.local/bin`, and `./install.sh --uninstall` removes it while keeping your wallet in `~/.srnet`.
4. **First run.** `srnet key new` makes your encrypted wallet, `srnet doctor` checks your setup, and `srnet ui` opens the app.

macOS works for SNApps, mail, files and the wallet, but the install steps cover Ubuntu only, and routing traffic through an exit is Linux-only. Windows is not supported in this release.

The app has three tabs: **SNApps**, **Exits** and **Wallet / Top up**. The SNApps window itself starts with `srnet snapps --config client.json`. On Linux the installer also registers `srnet://` links, so clicking one opens the join sheet in the SNApps window. On macOS, `srnet open <link>` does the same.

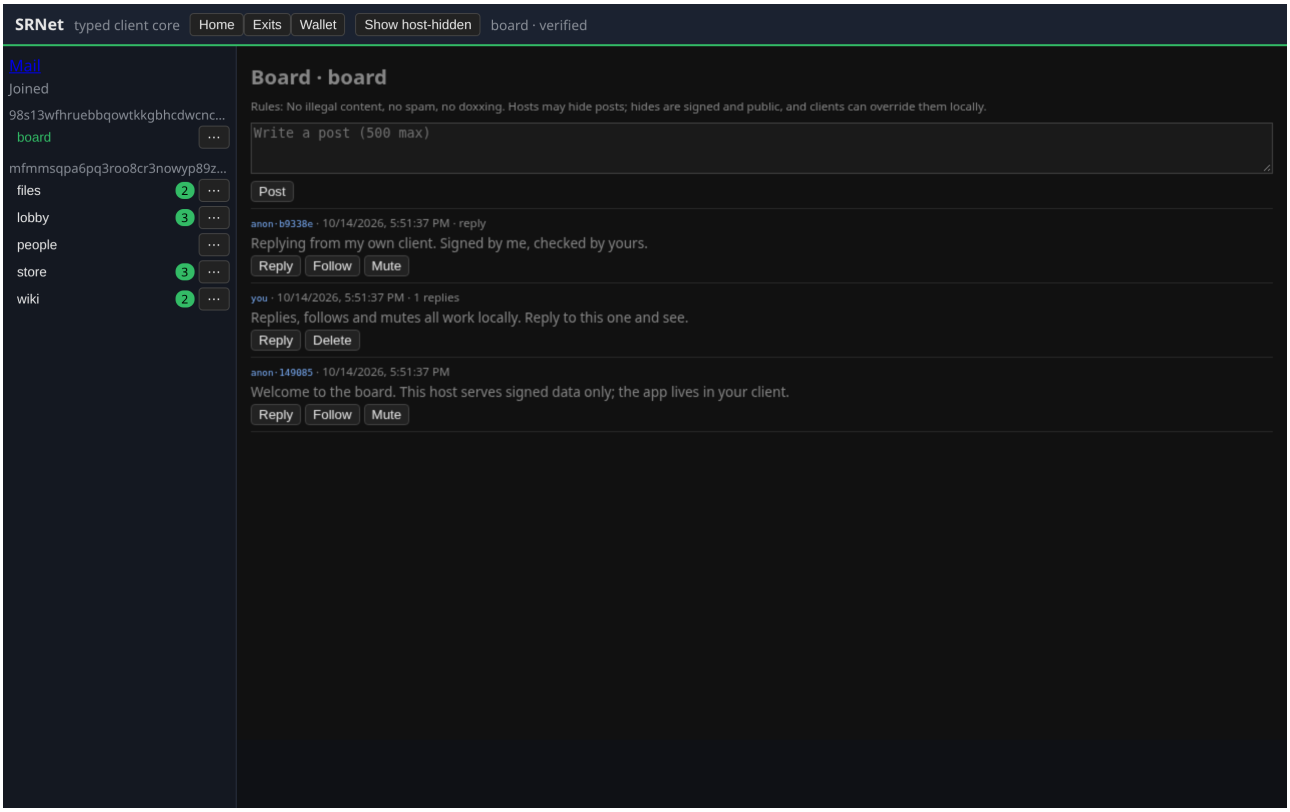


Figure 1. The SNApps window. The ribbon and the verified status line across the top, Mail and your joined collections in the sidebar, the active template in the middle, and the reminder about keys at the bottom.

Every typed SNApp lives inside the same plain, dark shell. The ribbon across the top holds just three buttons: **Home**, where you join new collections, **Exits** and **Wallet**. Next to them sits a **Show host-hidden** toggle, so you can see anything a host has chosen to hide and judge it for yourself. On the right, the status line names the collection you are reading and confirms it, as in board · verified.

The sidebar on the left starts with **Mail** and its unread count. Below it, under **Joined**, sit the collections you belong to, grouped by the host that serves them, each with its own unread count and a ⋯ menu. Section 4 covers the sidebar in detail.

The middle of the screen is the template. It runs in a **sandbox**: it receives verified records, it can propose a new one, and that is all. The footer says so on every screen: the core verifies every record; templates run sandboxed with no network and never see your keys; only the core signs, after you confirm. That sentence is the trust boundary the whole design rests on.

3. The apps, one by one

3.1 Message board

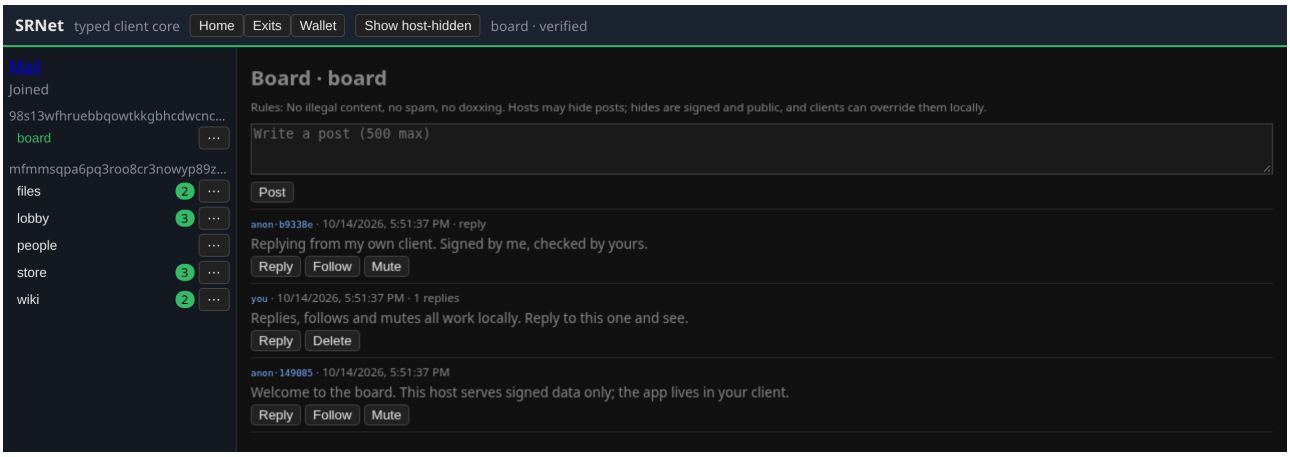


Figure 2. The board with its rules line, the compose box and three signed posts, one of them a reply.

The board opens with the host's rules in one line, a compose box with a 500-character limit, and the posts below, newest first. Each post shows its author's key tag and time. Your own posts are marked **you** and carry **Reply** and **Delete**; everyone else's carry **Reply**, **Follow** and **Mute**. Posting never goes straight to the host. The template proposes the post, and the core signs and sends it only after you confirm. Follows and mutes stay on your device.

3.2 Chat room

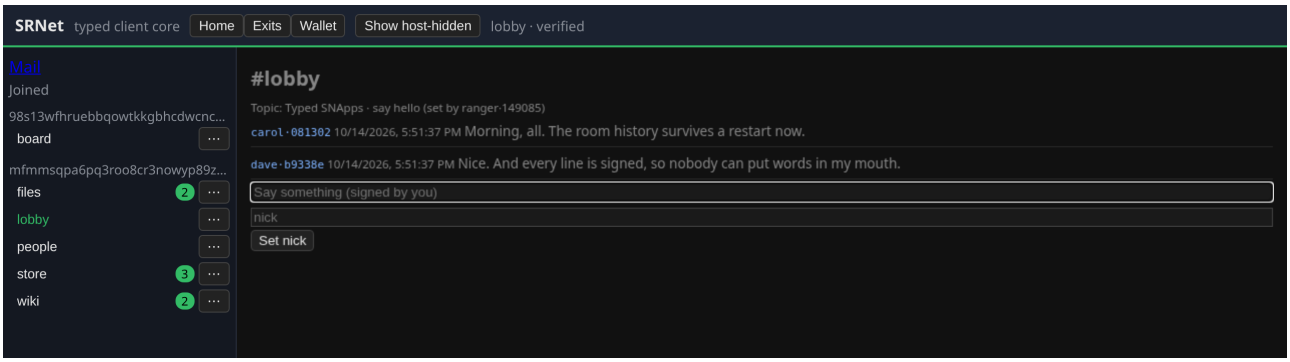


Figure 3. The #lobby room with its signed topic and two signed messages.

The lobby feels like any chat room: a topic line at the top, messages below, and a box to say something. The difference is underneath. Each line is signed by its author (the input box reads "Say something (signed by you)"), the topic is a signed record set by a room admin, and the room keeps a durable, verified history that survives a restart. Nicknames are only names. The key suffix after the dot, as in `carol·081302`, is what identifies the speaker.

3.3 Wiki

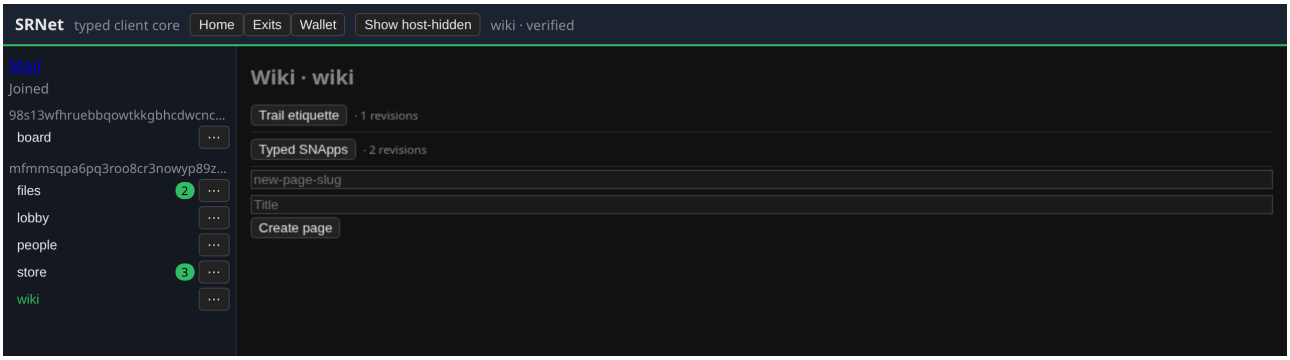


Figure 4. The wiki page list: "Trail etiquette" with one revision and "Typed SNApps" with two.

The wiki lists its pages with a revision count, and a small form below creates a new page from a slug and a title. Every revision is a signed record that points at its parent. Edits can be reverted, two edits on the same stale parent show up as a conflict, and an admin can protect a page. Only the signed revisions come from the host, never the page's look.

3.4 Store

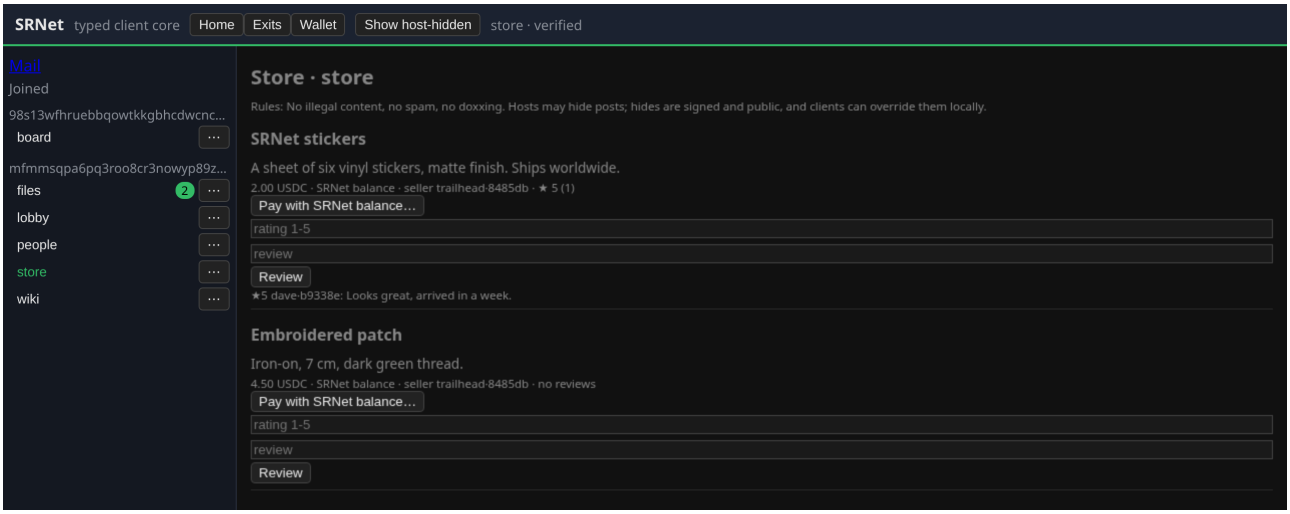


Figure 5. A store with two listings, each priced in USDC and paid from the SRNet balance, with a five-star review.

The store shows each listing with its title, description, price, the seller's key tag and its rating. Prices are in USDC and every listing carries one button: **Pay with SRNet balance…**. Below each listing you can leave a signed rating and review.

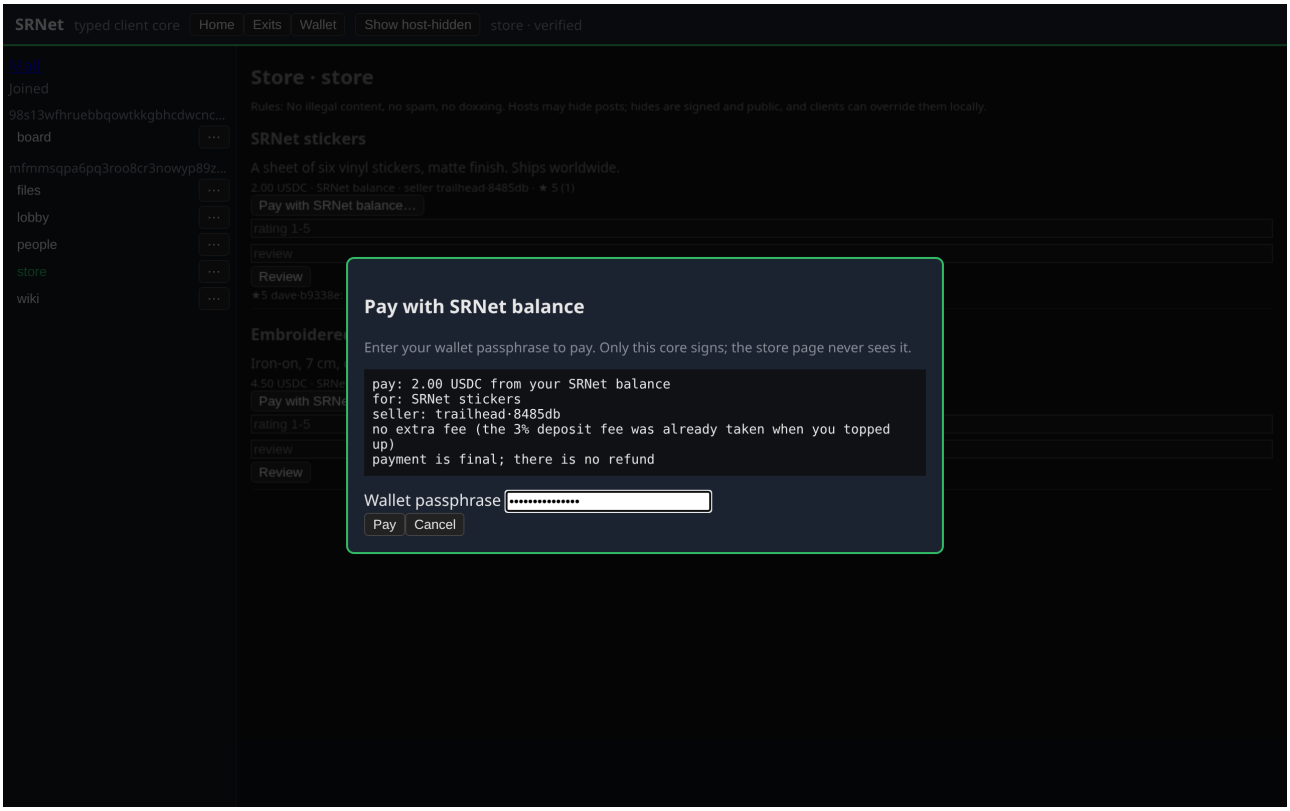


Figure 6. Pressing the button opens the core's own payment sheet. The store page never sees the passphrase and never signs.

The button does not pay. It asks the core, and the core opens its own sheet on top of the page: the amount, the item, the seller, a reminder that the deposit fee was already taken at top-up, and that payment is final. You type your wallet passphrase into the core's sheet and press **Pay**. The core then sends one payment from your SRNet balance to the seller's vendor id and posts a signed receipt to the store, which the seller's client checks on-chain before shipping. Section 7 covers the balance itself.

3.5 File drop

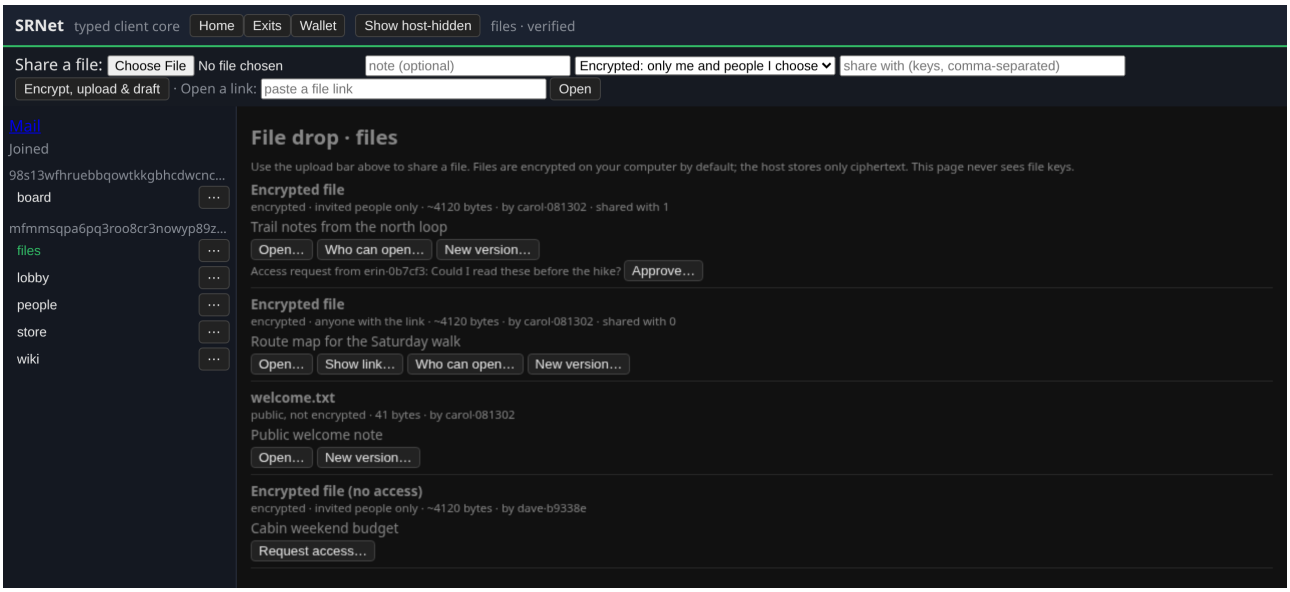


Figure 7. The file drop with all three sharing modes, an access request waiting for approval, and a file you can ask to open.

The file drop now encrypts by default. The upload bar at the top belongs to the core, not the template. You choose a file, add an optional note, pick who can open it, and press the encrypt-and-

upload button. The core encrypts the file on your computer before anything leaves it, and the host stores only ciphertext.

The list in Figure 7 shows each case. Your trail notes are encrypted and shared with one invited person, and erin has asked for access with a short note and an **Approve…** button beside it. The route map is encrypted and opens for anyone who holds its link. The welcome note is public on purpose. At the bottom sits dave's cabin budget, which you were not invited to, so it offers **Request access…** instead of **Open…**. Encrypted entries show no file name, because the name travels inside the encrypted file.

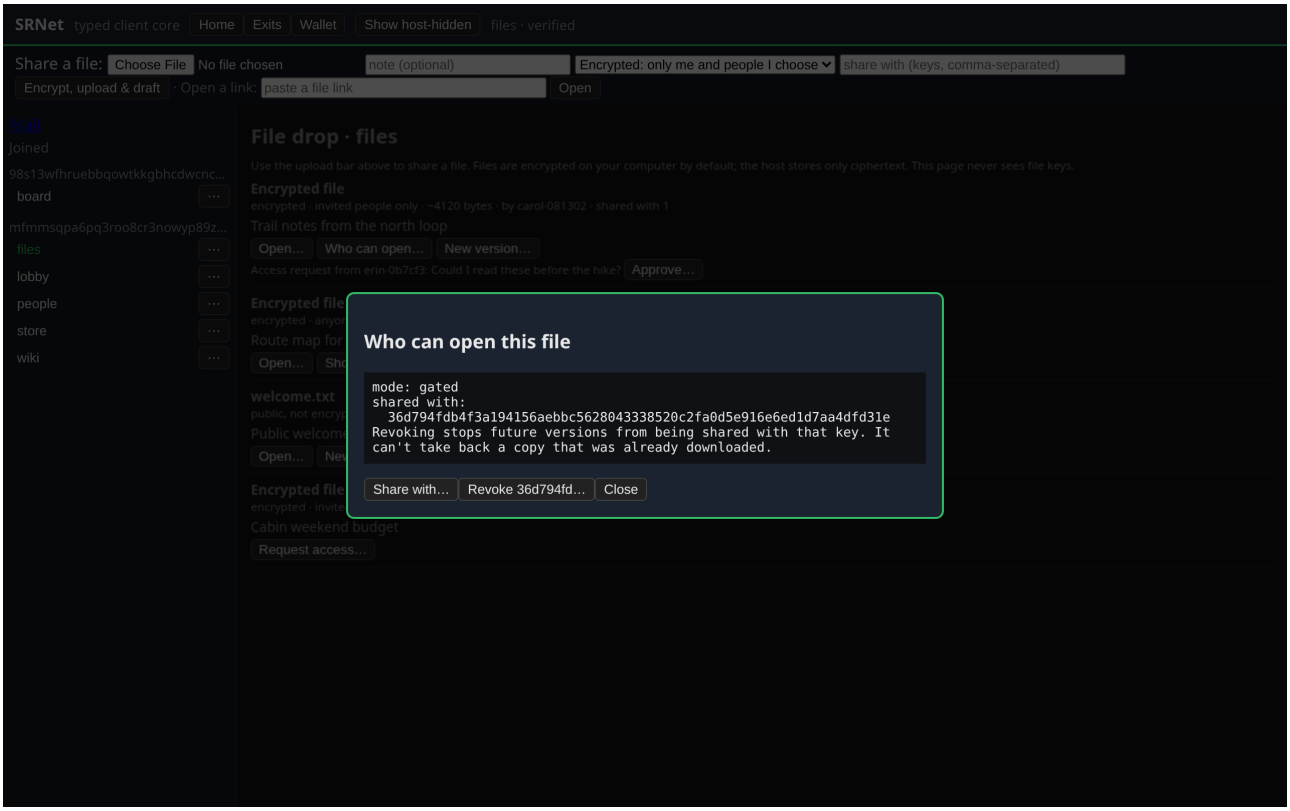


Figure 8. Who can open this file: the sharing mode, the invited key, and buttons to share with someone new or revoke.

Who can open… brings up another core sheet. It shows the mode, every key the file is shared with, a **Share with…** button for adding someone by their profile key, and a **Revoke** button for each person. Section 6 explains what happens underneath.

3.6 People

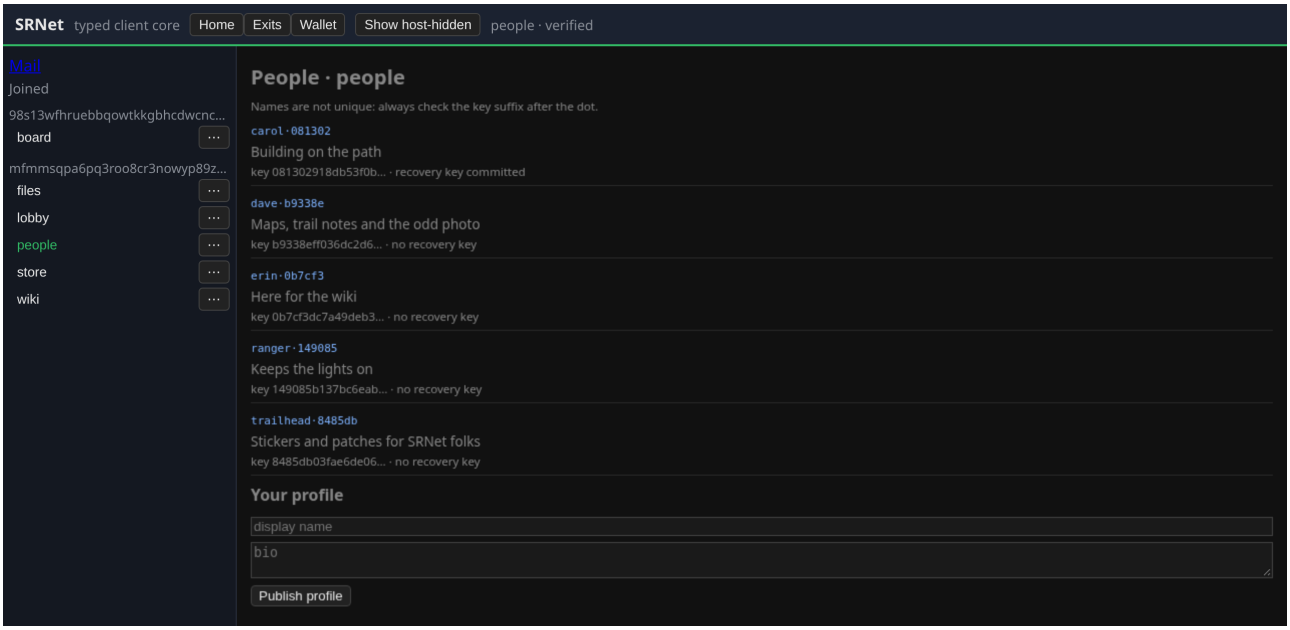


Figure 9. The people directory: five signed profiles, one with a recovery key committed.

The people tab shows signed profiles. The profile "carol" has a short bio, "Building on the path", and the note **recovery key committed**: the profile carries the hash of a recovery key made in advance, so if the posting key is ever stolen, the owner can take the identity back. Names are not unique, so the client always shows `handle·<key prefix>`, and a look-alike handle cannot hide a different key. Each profile also carries the owner's public encryption key, which is how others share encrypted files with them. A small form below lets you publish your own profile.

3.7 Exit board

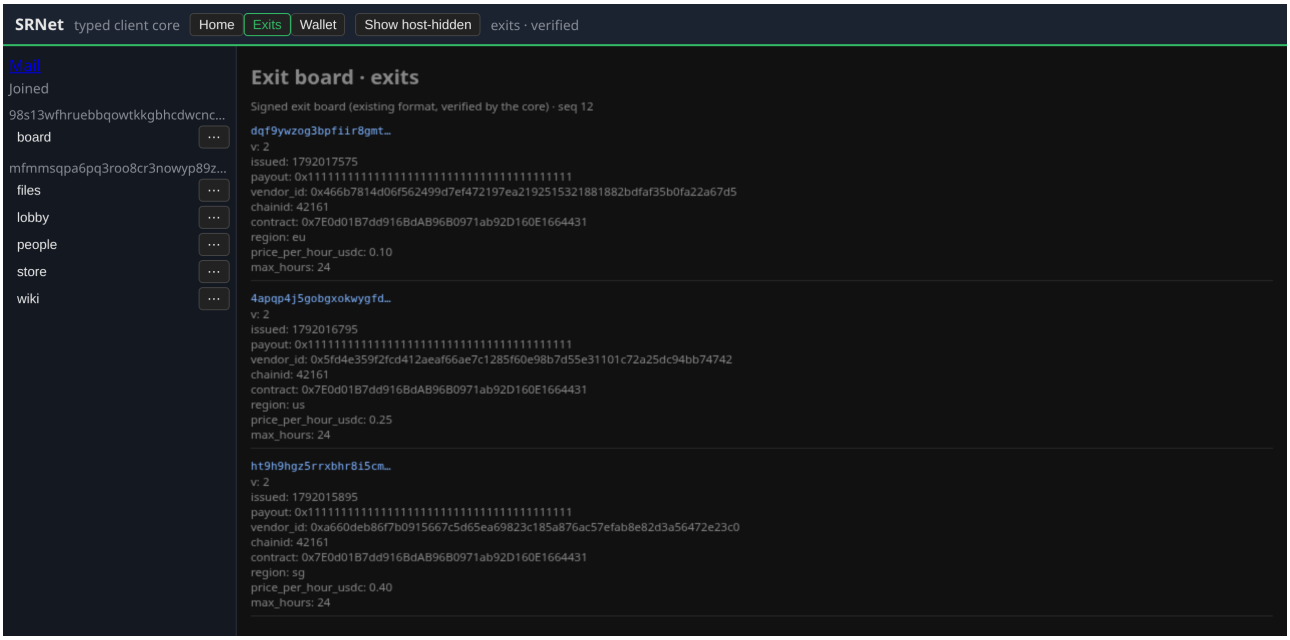


Figure 10. The signed exit board read by the typed client, with three seller listings.

The **Exits** button in the ribbon opens the signed exit board, read in its own format, and lays out each listing plainly: version, issue time, payout address, vendor id, chain, contract, region, hourly price in USDC and the longest lease the seller offers. The exit board was a typed SNApp in all but name before typed SNApps existed, and the client reads it through a small adapter. Section 8 shows the Exits tab where you buy.

3.8 Directory

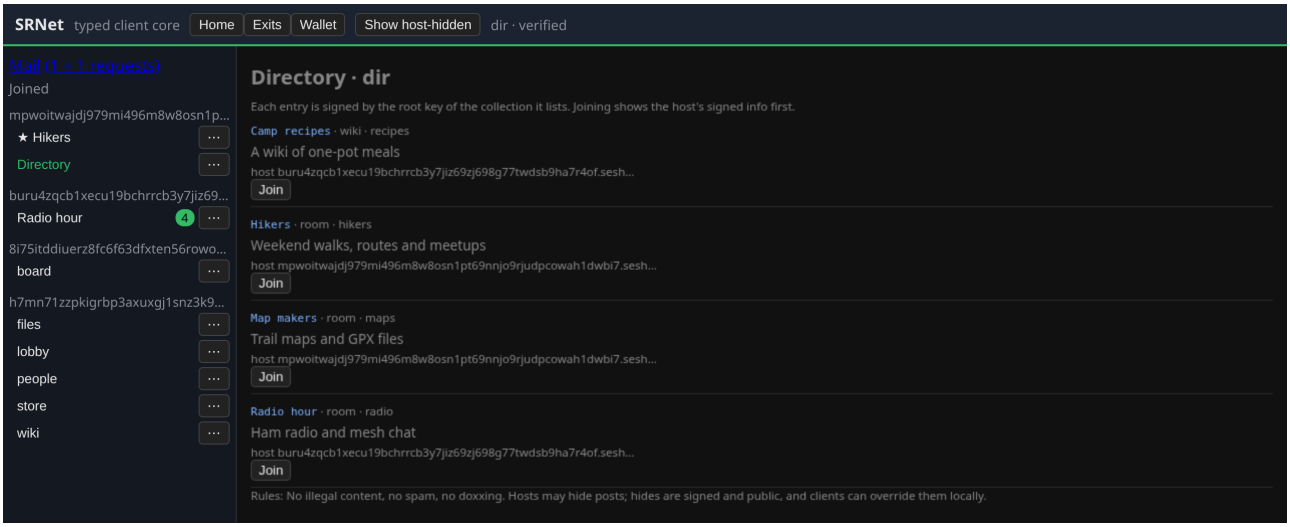


Figure 11. A directory of communities on two hosts. Each entry is signed by the root key of the collection it lists, and each one has a Join button.

A directory is a typed collection that lists other collections: a chat room here, a wiki there, each with a title, a short description, its type and the host that serves it. Hosts opt in. An entry counts only if it is signed by the **root key of the collection it lists**, so nobody can list your community but you, and nobody can attach your title to someone else's host. The client drops any entry that fails that check before the template sees it.

Join on an entry does not join anything by itself. It opens the same join sheet as a pasted link, described next, and the host is checked again before anything is saved.

4. Joining communities on other hosts

Anyone can run a host, so the rooms, boards and wikis you care about will live on many different machines. The client keeps them together in one sidebar and checks each one against the key you first trusted.

4.1 Invite links and the join sheet

An invite link looks like this:

```
srnet://<host>.sesh/room/hikers#root=<root key>
```

It names the host, the type, the collection and the collection's **root key**. Paste a link into **Home**, click one anywhere your desktop opens links, or press **Join** in a directory. A bare `.sesh` address works too: the client asks the host what it offers and lists each collection with its own **Join** button.

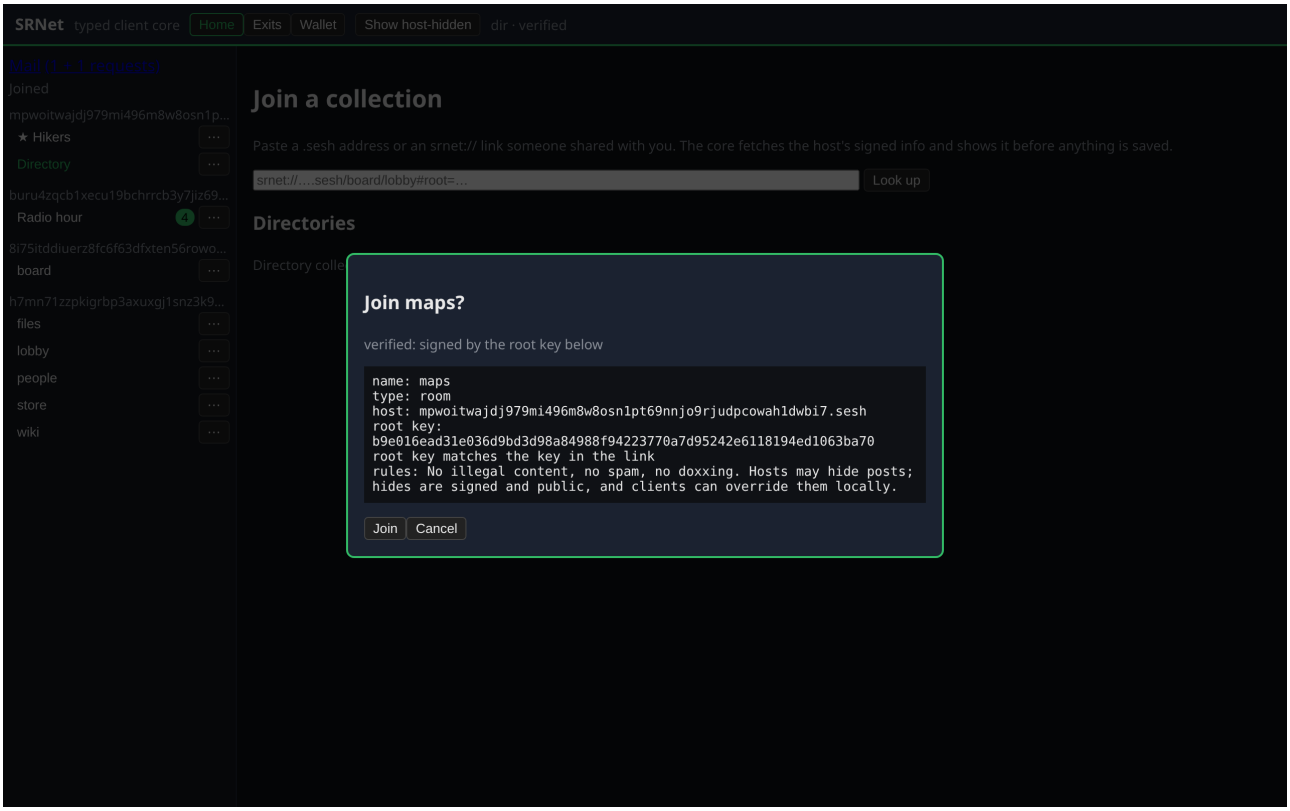


Figure 12. The join sheet. The core fetched the host's signed details and checked them against the root key in the link. Nothing is saved until you press Join.

The core contacts the host, fetches its signed delegation and checks the signature before it shows you anything. The **join sheet** lays out the collection's name, type, host and root key, whether that key matches the one in the link, and the host's rules. A link whose root key does not match what the host presents is refused outright. Nothing is stored until you press **Join**, and joining the same collection twice simply opens the one you have.

4.2 Root-key pinning

The first time you join a collection, the client **pins** its root key. Every later sync checks the host against that pin. If the host ever presents a different root key, through a takeover or a rotation, the client stops syncing that collection, marks it with ⚠ in the sidebar and shows a warning that names both keys. Your cached copy stays as it was.

If you expect the change, for example because the operator announced a new key, you can **re-trust** it. The client asks you to type trust the new key, wipes the cached copy and syncs again from scratch under the new key. Links shared by the host carry the root key, so a link you received from someone you trust pins the right key from the very first join.

4.3 The sidebar

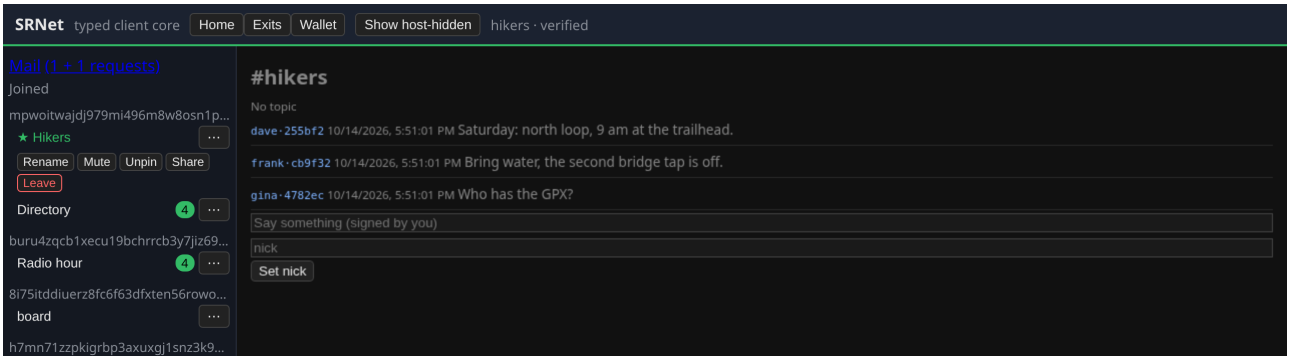


Figure 13. The sidebar: Mail with its unread count, then joined collections grouped by host. Hikers is pinned to the top and its menu is open; Directory and Radio hour show unread counts.

The sidebar groups your collections by host, with pinned ones first. A green badge counts new posts since you last opened a collection; your own posts and profile updates never count. The ⋯ menu on each row offers:

- **Rename**, to give the collection your own name for it.
- **Mute**, to keep it but stop its unread count.
- **Pin**, to move it to the top.
- **Share**, which shows an invite link with the host, the collection and its root key, ready to pass on.
- **Leave**, which removes it and deletes its cached copy.

Everything you join, rename, mute or pin comes back after a restart, with no config file to edit.

4.4 Petnames

Names on SRNet are not global, and the client does not pretend they are. Instead it keeps **petnames**: the names you give to collections and people, stored only on your device. Rename a room to "Saturday walkers" and that is what your sidebar shows, while the link and the root key underneath stay the same. In Mail, the name you give a contact is the name you see on their messages. A shared mail address can carry a suggested name, and the client uses it only until you choose your own. A petname never changes which key a collection or a person is.

5. Mail

SRNet mail is store-and-forward mail for people who are not online at the same time. You write a message, the client seals it to your contact's key, and one or more **mail hosts** keep it until their client picks it up. The hosts can store it, count it and delete it. They cannot read it, and they cannot see who sent it.

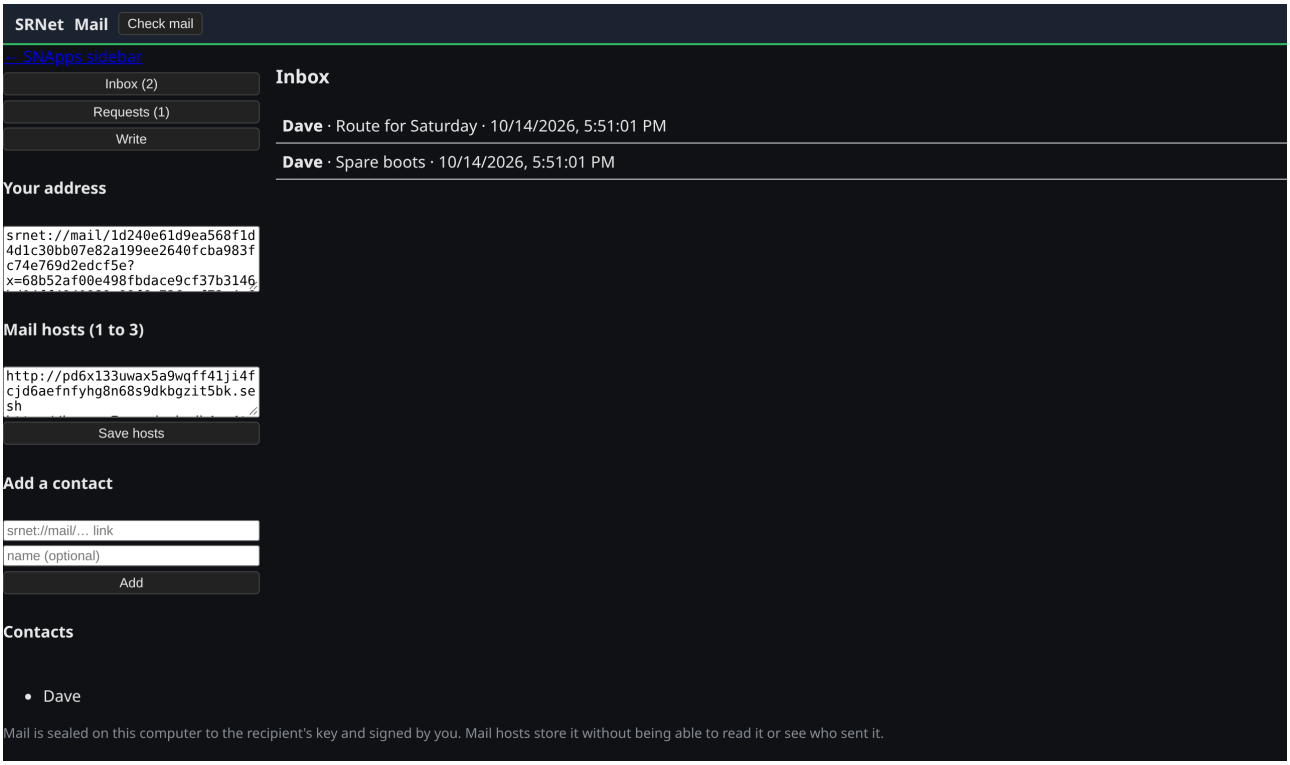


Figure 14. The Mail page. Inbox and Requests with their unread counts, your address, your mail hosts and your contacts on the left; the Inbox on the right.

5.1 Your address

Your mail address is a `srnet://mail/…` link. It carries your public identity key, your public encryption key and the one to three mail hosts you chose, all signed by you, so a host cannot be swapped into it. The Mail page shows it under **Your address**, ready to copy. Give it to the people you want to hear from, and paste theirs into **Add a contact**, with a name of your choosing. You pick your mail hosts under **Mail hosts**: any host running the mail role works, and listing two or three means mail still reaches you when one of them is down.

5.2 Sealed and signed

Every message is **signed by its sender** and then **sealed** to the recipient's encryption key with a fresh one-time key, using X25519 and ChaCha20-Poly1305. The seal is bound to the recipient and the message id, so a host cannot move a message to another inbox or relabel it. Inside the seal, the message is padded to a power-of-two size between 1 KiB and 64 KiB, so the host learns only a rough size.

When a message arrives, the client opens the seal, checks the sender's signature and shows **signature verified** next to the sender's name. A message with a bad signature, or one that is not for you, is rejected and never shown.

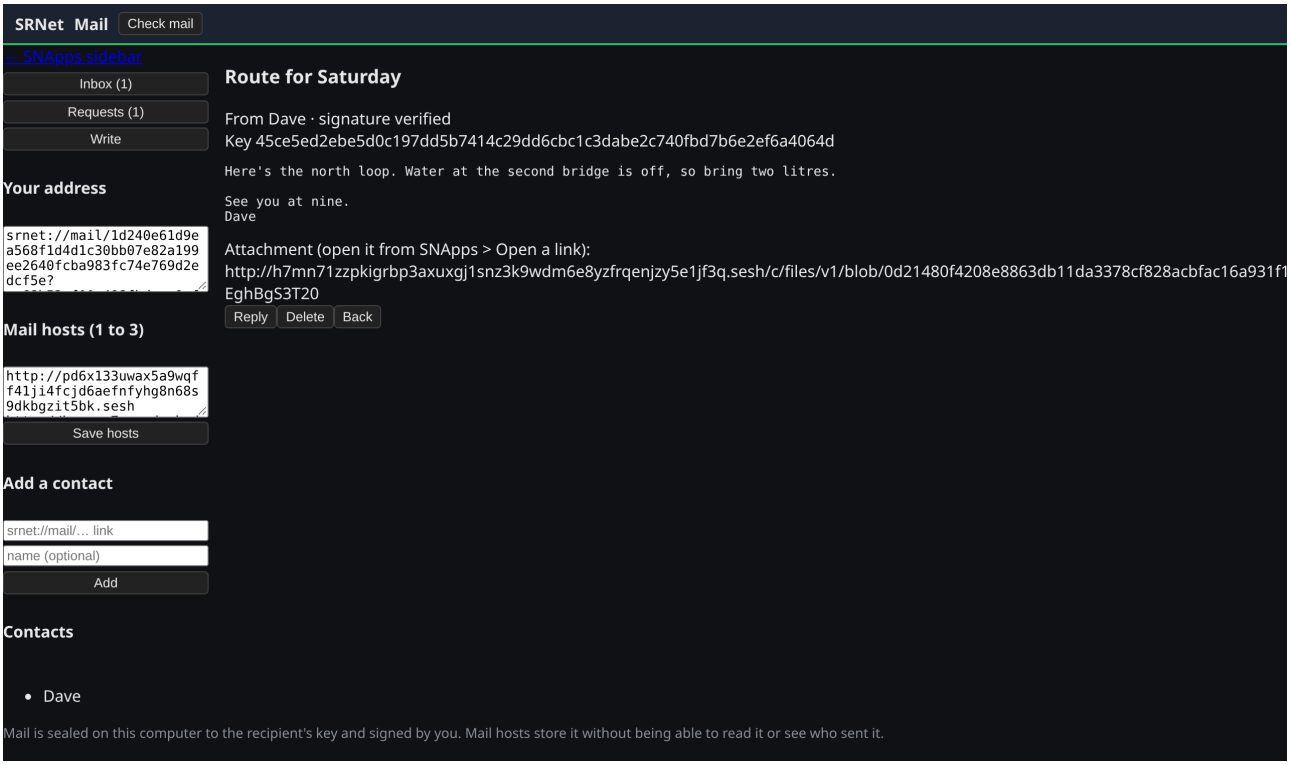


Figure 15. An opened message: the sender's petname and key, the verified signature, the body and an encrypted attachment, with Reply, Delete and Back.

5.3 Inbox and Requests

Mail from your contacts lands in the **Inbox**. Mail from anyone else lands in **Requests**, a small separate pile with tight limits: by default 32 messages of up to 16 KiB each, and no more than 20 an hour. A host can raise these with `--requests-max-msgs` and `--requests-per-hour`. Opening a request shows **Accept sender**, which adds them to your contacts and moves their mail to the Inbox.

The host does the sorting without learning who wrote what. Your client gives each of your hosts a short **contact tag** for every accepted contact, computed from a secret only you and that contact share, and different on every host. A sender's client attaches the tag, and the host sees only whether it matches. It never sees a name or a key.

5.4 Delivery and pickup

The sender's client delivers each message to **every** host on the recipient's address, so one host going down loses nothing. When you press **Check mail**, the client asks each of your hosts for its waiting mail. To prove the inbox is yours, it signs a fresh one-time challenge from the host with your key. There are no passwords and no shared tokens.

Pickup is **fetch, then ack**. The host hands over the sealed messages but keeps them. The client saves them to disk first, then tells the host which ones it has, and only then does the host delete them. If the client stops halfway, the next check simply sees the same messages again, and the client drops the duplicates by message id, along with the extra copies from your other hosts. Mail that is never picked up expires after 7 days by default, or up to 30 if the sender asks.

5.5 Attachments

Attach takes files straight from your computer. The core encrypts each file on your machine, up to the file drop's size limit (25 MiB by default), stores the ciphertext on a file drop you have joined, and puts the file's link, with its key, inside the sealed message. Only the people you send the message to get the key, and the file is not listed anywhere on the drop. Recipients open attachments from **Open a link** in the SNApps window, and the core decrypts them there.

5.6 What a mail host sees

A mail host sees which inbox a message is for, its padded size and when it arrived. It does not see the sender, the subject, the body or the attachments. Each host is open to new inboxes by default, up to 1,000, and its operator can close sign-up at any time. Existing inboxes keep working either way.

From the command line, `srnet mail` covers the same ground: `hosts`, `address`, `contact`, `send`, `fetch`, `list`, `read`, `accept` and `name` for petnames.

6. Files, encrypted by default

Every upload gets its own fresh 256-bit key and is encrypted in 64 KiB chunks with **ChaCha20-Poly1305**. Each chunk is bound to its position and to the file header, so chunks cannot be reordered, dropped, cut short or extended. The file's name and type go inside the encryption, and the whole thing is padded up to the next power of two, at least 4 KiB, so the host learns only a rough size bucket. That is why the encrypted entries in Figure 7 all read about 4,120 bytes. Each file drop host sets its own largest file size, 25 MiB by default.

You choose one of three modes per file:

- **Only me and people I choose** (the default). The file key is wrapped to your own encryption key and to each person you invite, using the encryption key in their SRNet profile. You can add people later with **Share with…**, and when someone sends an access request, **Approve…** wraps the key to them.
- **Anyone with the link.** The key rides in the link after `#k=`. Browsers and the client never send that part of a link to the host, so the host serves the file without ever seeing its key.
- **Not encrypted.** An explicit choice for content meant to be public and indexable. The client asks you to confirm before it uploads.

Revoking someone stops them getting future versions: **New version…** re-encrypts the file under a new key and shares it with everyone except the people you revoked. A copy someone already downloaded stays theirs. Templates never see keys, wraps or links. The core decrypts files itself and shows links and access lists only in its own sheets.

7. One way to pay: your SRNet balance

Every paid action on SRNet uses the same balance and the same flow. Exits, store items, and later paid files and rooms all say **Pay with SRNet balance**. From the command line, `srnet pay --vendor-id V --amount N` pays any SRNet seller the same way.

The balance lives in the **SRNetBalance** contract on Arbitrum One at `0x7E0d01B7dd916BdAB96B0971ab92D160E1664431`, with its source verified on Sourcify. The contract is unaudited, so keep amounts small.

- **The confirmation is always the core's.** The core or the command line shows its own prompt, and only then signs. A template can ask for a payment; it can never build, sign or send one.
- **Fees are simple.** Topping up takes a 3% deposit fee. A phone-wallet top-up also pays the relayer a flat fee, 0.05 USDC by default and never more than 1 USDC. A payment itself adds nothing.
- **Payments are small and final.** Each payment is capped at 20 USDC, starts at once and cannot be cancelled or refunded.
- **Sellers set up once.** A store seller runs `srnet vendor --shop-key <key>` and puts the vendor id in their listings. The seller's signature on the listing ties the store key to that vendor id.

- **Gas.** Topping up needs no ETH at all. Spending from the balance is one small transaction from your wallet, so keep a little ETH there for gas.

Store payments from the SNApps window switch on for Arbitrum One with one line in `client.json`: `"payments": {"allow_mainnet": true}`.

In the screenshots, the wallet received 10 USDC from a phone wallet, was credited 9.65 after the 3% fee and the 0.05 relayer fee, and then paid 2.00 for the stickers in Figure 6, leaving 7.65.

8. Exits and the Wallet tab

8.1 Buying exit time

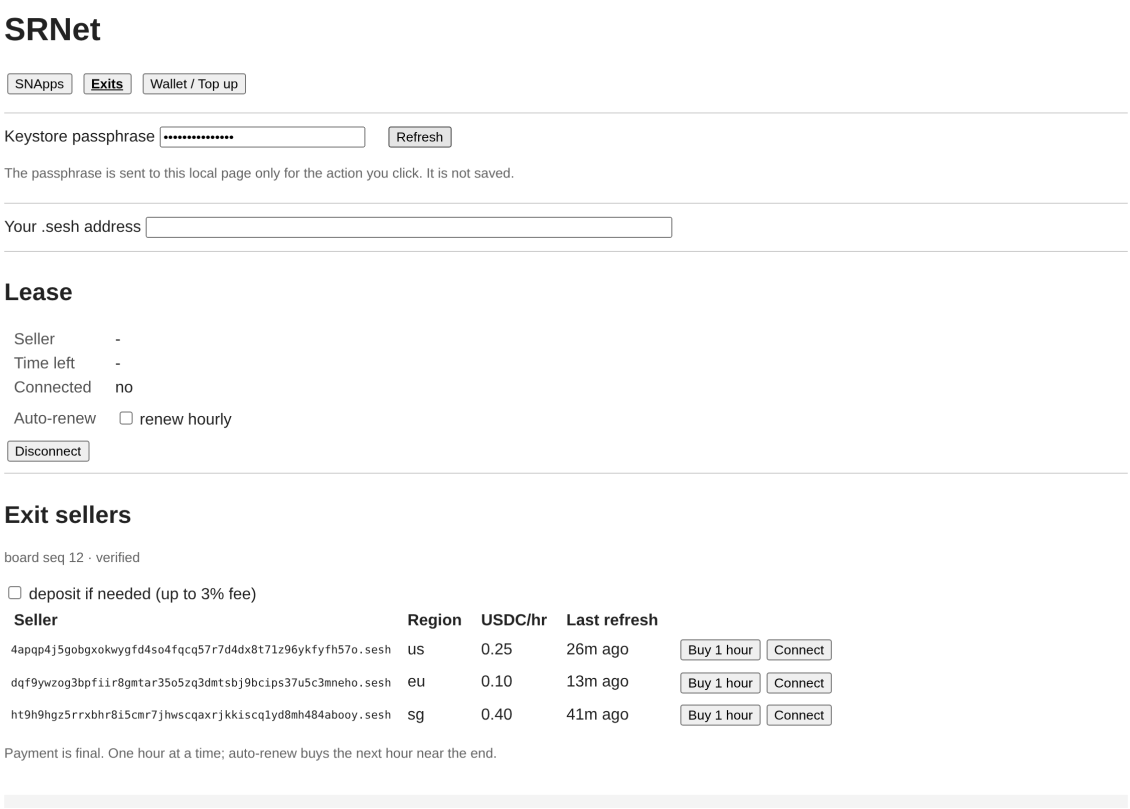


Figure 16. The Exits tab: your lease at the top, sellers from the signed exit board below, each with Buy 1 hour and Connect.

An **exit** is a Session Router node that forwards your traffic out to the open internet. Independent sellers run them, and you rent one by the hour. The Exits tab lists sellers from the signed exit board with their region, price per hour and last refresh, and gives each one **Buy 1 hour** and **Connect**. The board comes from three `.sesh` names with failover, and the client keeps its last verified copy. The **Lease** panel above shows your seller, the time left, whether you are connected, and an **Auto-renew** switch that buys the next hour near the end of the current one. The keystore passphrase is used only for the action you click and is never saved.

The rules of a lease fit in a few lines. You prepay whole hours from your SRNet balance, at most $20 per lease, and payment is final once a lease starts. Buying one hour at a time keeps the most you can lose to about one hour. The first time you buy or connect, the client explains what an exit operator can see and asks you to type `I UNDERSTAND`. Routing your traffic through an exit with `srnet connect` uses nftables and runs on Linux. SNApps, mail, files and the wallet also work on macOS. Windows is not supported in this release.

Your claim reaches the seller by itself. After you buy, the client sends your signed claim straight to the seller over Session Router, to the address in the seller's own signed listing, and waits for the seller's acknowledgment. Delivery retries with backoff, and the seller ignores duplicates, so `srnet deliver` can resend it at any time. Sellers receive claims with the seller pack's claim listener.

8.2 Topping up from a phone wallet



Figure 17. Wallet / Top up: your address and balances, and a QR code any phone wallet can scan.

The Wallet tab shows your wallet address, the USDC in your wallet, your SRNet balance and your ETH for gas. To top up, enter an amount and press **Show QR**. The QR code is a plain USDC payment request to **your own** SRNet wallet on Arbitrum One, so any phone wallet can scan it and send. No ETH is needed anywhere in this step.

When the USDC arrives, press **Deposit when it arrives**. The client signs a gas-free approval, and a relayer moves the USDC into your SRNet balance. The relayer pays the gas and takes its small flat fee; it cannot send your funds anywhere else or charge more than you signed. You can set a relayer in the Wallet tab or with `srnet topup --relayer URL`, and anyone can run one with `srnet-relayer`. The same flow works from a terminal with `srnet topup --amount 10`.

9. The locked-down HTML viewer

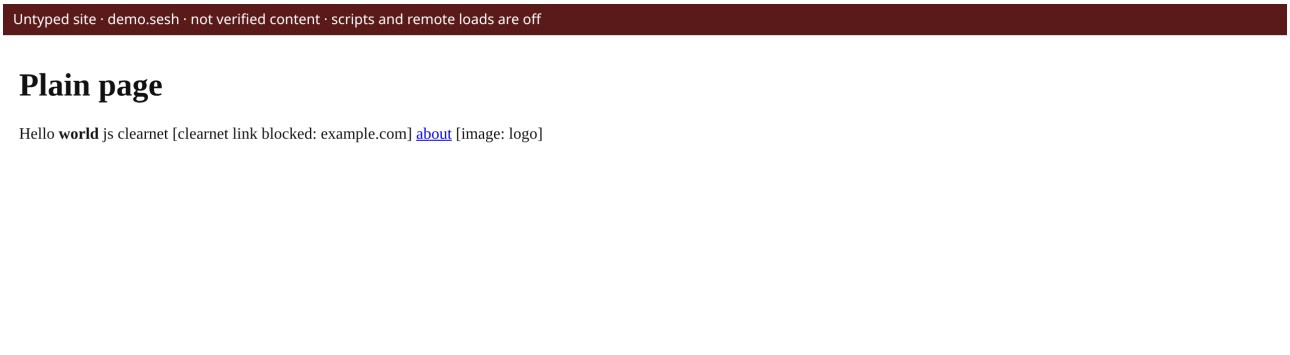


Figure 18. A deliberately hostile test page in the locked-down viewer. The script and remote image are gone, the clearnet link is blocked, and a permanent banner marks the site as untyped.

Some sites will never fit a type, and that is fine. They open in the viewer. The test page above, `demo.sesh`, carried a script, a clearnet link and a remote image. What remains after the allow-list cleaner has done its work is plain text and a same-name link. The script is gone, the clearnet link is replaced by `[clearnet link blocked: example.com]`, and the image shows only as `[image: logo]`.

A red banner across the top reads Untyped site · demo.sesh · not verified content · scripts and remote loads are off. It cannot be dismissed, so you always know you have stepped off the typed path. The viewer is a fallback, not a second browser. A site that wants a richer interface defines a type.

10. Running your own host

You can carry a piece of the SRNet community yourself: a message board, store, wiki, chat rooms, file drop, profile directory, directory of communities or mail host. You do not write or ship app code. The apps are in the client, and your server only stores and serves signed or sealed data.

The **SRNet host pack** lives in the SRNet-TypedSNApp-HostPack repository on GitHub. On a Debian or Ubuntu server, install `git`, `python3` and `python3-cryptography`, clone the repository and run `./bin/srnet-host` from its folder. Clone it on your offline machine as well, for the root key. The host guide, `docs/HOST-GUIDE.md`, covers every step.

The pack uses two keys:

Key	Lives on	Does
Root key	a separate machine that stays offline	Signs for your server for up to 90 days at a time. Its public half, the **root pin**, travels in every invite link, and clients pin it on first join.
Host key	the server	Signs the day-to-day data. Made for you. If the server is ever compromised, you stop renewing it.

Setup takes about ten minutes on a Linux server with Python 3.10 or newer:

1. On the offline machine, run `srnet-host root init` once and write down the root pin.
2. On the server, run `srnet-host init` with a type and a name. This makes a host key and a `request.json`.
3. Carry `request.json` to the offline machine and sign it with `srnet-host root sign request.json --days 90`. Check that the host key shown matches the one the server printed.
4. Back on the server, install the signed delegation with `srnet-host install-delegation`. On a systemd server, the pack sets up a locked-down system service under its own user. Elsewhere,

`srnet-host run` runs the host in the foreground. Either way it prints your connect info and an invite link for each collection.

5. Check on it with `srnet-host status`, and renew every 90 days or sooner. `status` warns you two weeks ahead.

The types, or roles, are:

Role	Serves
`board`	a message board
`store`	a store; buyers pay from their SRNet balance in their own client
`wiki`	a wiki
`rooms`	one or more chat rooms (`--room` adds another)
`files`	an encrypted file drop
`people`	a profile directory
`directory`	a directory of communities
`mail`	a mail host

Your host's own `.sesh` name. During `init` you can **make** a new `.sesh` key for your host, **import** a key you already have (for example one from a vanity-name search), or skip and add one later with `srnet-host sesh`. Imported keys are checked and copied unchanged, and the secret is never printed. The host itself listens only on loopback. When you are ready, you start a separate Session Router instance for its name from the config the pack wrote, and the host is reachable at `http://<name>.sesh`.

Sharing it. `srnet-host link` prints the invite links for your collections, each carrying your `.sesh` name, the collection and your root pin. Those links are all anyone needs to reach the join sheet in Section 4.

Listing in a directory. On the offline machine, `srnet-host root directory-entry --link <your link> --title …` signs an entry for one of your own collections with its root key. `srnet-host directory-submit` posts it to any directory from an online machine. To run a directory yourself, use the `directory` role.

Running a mail host. `srnet-host init --type mail` sets up a mail host. It is **open** by default, so anyone can create an inbox, up to 1,000. `srnet-host mail --closed` stops new inboxes while existing ones keep working, `--open` turns sign-up back on, and `--max-inboxes` changes the cap. `srnet-host link` prints the URL people add to their mail hosts.

Keeping it current. To move to newer code, update your copy of the pack (for example with `git pull`), then run its `srnet-host upgrade`. It swaps in the new code, keeps your keys, data, delegations and `.sesh` name, keeps the previous code alongside, and restarts the service. `srnet-host backup` writes an archive of your data and host key, never the root key.

You are responsible for what you host under your local laws.

11. Under the hood

- **Signed records.** Every post, line, revision, listing, file entry, profile and directory entry is a small JSON record signed by its author's key. Change one byte and the signature fails. Bodies are plain text, never HTML.
- **Hash chain and checkpoints.** Records in each collection form an append-only log linked by hash, and the host signs checkpoints over it. Your client pins the collection's root key, refuses a

checkpoint older than one it has already seen, and treats two different heads at the same position as proof of a dishonest host. If a host fails a check, the client moves on to the next host or mirror.

- **Pinned roots.** The first join pins a collection's root key. A changed key stops syncing until you re-trust it on purpose.
- **Spam limits.** Each post carries a small proof-of-work puzzle, and hosts apply rate limits. New keys get a slightly harder puzzle. Mail from strangers goes to a small, rate-limited Requests pile.
- **Open moderation.** A host can hide posts, but hides are signed and public, and you can choose to see past them.
- **Sealed mail.** Messages are signed by the sender, sealed to the recipient, padded to a size bucket and delivered to every one of the recipient's hosts. Inboxes open with a signed one-time challenge, and hosts delete mail only after the client confirms it saved it.
- **Template sandbox.** Templates run in a sandboxed frame under a strict content policy, with no network and no keys. They can only propose. The core signs, after you confirm.
- **Template registry.** A signed registry pins which templates may run, offers updates for you to approve or reject, refuses rollback to older versions, and keeps a withdrawal list for known-bad releases.
- **Keys and payments stay in the core.** File keys, links, access lists, mail keys, passphrases and payment prompts never reach a template.

12. Getting started

To explore typed SNApps, install the client, run `srnet key new` and `srnet doctor`, then `srnet ui` and open the SNApps tab.

To join a community, paste its invite link into **Home**, or press **Join** in a directory, check the join sheet and press **Join**.

To start using mail, open **Mail**, set one to three mail hosts, and share your address with the people you want to hear from.

To fund your balance, open **Wallet / Top up**, scan the QR code with your phone wallet, send USDC on Arbitrum One, and press **Deposit when it arrives**.

To rent an exit, open the Exits tab, buy one hour from a seller and press **Connect**. Start with one hour and small amounts.

To run a host, clone the host pack repository and follow `docs/HOST-GUIDE.md`. Keep your root key on a machine that stays offline.

See you on the path.